4,786개 노드가 말하는 인간-AI 인지 협업의 실증적 발견

삼부작의 두 번째 — 시스템에서 발견으로의 귀납적 보고

SECTION OVERVIEW

11개 실증적 발견

Finding 0 — 무엇을 만들려 했고, 실제로 무엇이 만들어졌나

Finding 1 — 가장 많이 저장되는 것은 사실이 아니라 판단이다

Finding 2 — 실패를 기록하면 시스템이 달라진다

Finding 3 — 타입 체계는 미리 설계할 수 없다

Finding 4 — 토큰은 화폐다: Context as Currency의 실천적 교훈

Finding 5 — AI와의 협업에는 프로토콜이 필요하다

Finding 6 — 멀티 에이전트는 오케스트레이션 없이 카오스다

Finding 7 — 빼기가 더하기보다 어렵고 중요하다

Finding 8 — 자기 시스템을 측정하면 자기 사고가 보인다

Finding 9 — 온톨로지는 거울이다

Finding 10 — 만드는 행위 자체가 이해를 바꾼다

FINDING 00

무엇을 만들려 했고, 실제로 무엇이 만들어졌나

mcp-memory는 “외부 메모리 MCP 서버”로 시작했다. Claude Code 세션 사이에서 맥락이 소멸하는 문제를 풀기 위한 실용적 도구였다. SQLite에 노트를 저장하고 벡터 검색으로 꺼내는 — 흔한 RAG 서버.

그런데 지금 이 시스템은 4,786개 노드와 6,765개 엣지를 가진, L0-L5 6개 인지 계층과 25개 materialized 타입, 8개 relation 군, 12개 프로젝트 에코시스템으로 구성된 자기개선 인지 엔진이다.

> 의도한 것: 메모리 서버.
> 만들어진 것: 개인 메타인지 운영체계.

이 간극은 설계 실패가 아니다. 시스템을 운영하면서 “이것도 필요하다”는 발견이 반복적으로 일어났고, 그 발견들이 축적되어 설계를 바꿨다. Correction 타입이 사후에 추가된 것, schema 주석과 본문의 불일치, 에이전트 24→15 축소 — 이 모든 것이 “만들면서 배운 것”의 흔적이다. 의도와 결과의 간극 자체가 이 프로젝트의 가장 중요한 발견이다.

FINDING 01

가장 많이 저장되는 것은 사실이 아니라 판단이다

DB 타입별 분포에서 가장 놀라운 숫자: Decision 987개 — 전체의 20.6%.

이건 예상하지 못한 결과다. “지식 그래프”를 만들었으니 Observation이나 Insight가 가장 많을 것으로 예상했다. 그런데 가장 많이 저장된 건 판단이다. Knowledge base를 만들었는데 decision engine이 나왔다.

> 우리가 실제로 기억하고 다시 꺼내 쓰는 것은 “무엇이 있었는가”(사실)보다 “무엇을 결정했는가”(판단)다.

이것이 인간 인지에 대해 말하는 것: 우리가 실제로 기억하고 다시 꺼내 쓰는 것은 “무엇이 있었는가”(사실)보다 “무엇을 결정했는가”(판단)다. 그리고 이 판단에는 맥락이 붙어 있다 — 왜 그렇게 결정했는지, 어떤 대안을 버렸는지, 그 결정이 어떤 결과를 낳았는지.

두 번째로 많은 타입은 Tool(546개)이다. 이것도 의미 있다. 도구의 사용법, 설정, 트러블슈팅을 빈번하게 저장한다는 건 이 시스템의 사용자가 “알기 위해” 기억하는 게 아니라 “하기 위해” 기억한다는 뜻이다. 이 시스템은 관조적 지식이 아니라 실천적 지식의 저장소다.

FINDING 02

실패를 기록하면 시스템이 달라진다

Failure 141개. 대부분의 시스템은 성공만 남긴다. 노트 앱에 “오늘 실패한 것”을 적는 사람은 드물다. 그런데 실패를 1급 시민으로 저장했을 때 세 가지가 벌어졌다.

첫째, 같은 실패를 반복하지 않는다. Failure 노드가 세션에 주입되면 Claude가 “이전에 비슷한 실패가 있었다”고 경고할 수 있다.

둘째, 실패 패턴에서 구조가 보인다. 개별 실패는 우연처럼 보이지만, 모아놓으면 패턴이 드러난다. “문제 정의 생략 시 구현 후 재발견”(#5369)과 “부분 최적화가 전체를 깨뜨린다”(#5353)는 다른 사건이지만, 둘 다 “전체를 보지 않고 부분에 뛰어드는 습관”이라는 상위 패턴으로 승격될 수 있다.

셋째, Correction이 생긴다. 실패 패턴이 구조화되면 “다음에 이런 상황이 오면 이렇게 하지 마라”는 교정 규칙이 된다. 이 Correction이 recall 최우선으로 주입되면서 시스템은 자기교정 능력을 갖게 된다.

> 가장 구체적인 사례: node #5367, Perplexity 할루시네이션 사고. 이 실패가 기록되어 있기 때문에, 이후 API 기반 자동 게시 파이프라인에서 “URL 검증이 통과하지 않으면 게시하지 않는다”는 원칙이 설치되었다.

FINDING 03

타입 체계는 미리 설계할 수 없다

schema.yaml은 L0-L5에 걸쳐 51개 타입을 정의한다. 그러나 실제로 DB에 materialized된 타입은 25개. 나머지 26개는 아직 비어있다.

이건 과잉 설계가 아니다. 시스템이 아직 그 타입을 필요로 하지 않았다는 뜻이다. L4(Belief, Philosophy, Mental Model, Lens)와 L5(Axiom, Value, Wonder, Aporia)가 비어있는 건 시스템이 아직 그 깊이에 도달하지 않았다는 뜻이다.

반대로, Correction은 처음 schema에 없었다가 나중에 추가되었다. 시스템을 운영하면서 "자기교정 규칙을 저장할 타입이 필요하다"는 것을 발견했기 때문이다. 이건 미리 설계할 수 없었다. 운영해봐야 드러나는 필요.

> 온톨로지를 처음부터 완벽하게 설계하려 하지 마라. 최소한으로 시작하고, 필요가 발견되면 확장하라. 빈 자리를 예비해두되, 채우는 건 경험에 맡겨라.

FINDING 04

토큰은 화폐다 — Context as Currency의 실천적 교훈

200K 토큰 컨텍스트를 운영하면서 배운 가장 중요한 교훈: “더 많이 넣으면 더 좋다”는 거짓이다.

초기에는 가능한 한 많은 정보를 컨텍스트에 넣으려 했다. 에이전트 정의 24개, 스킬 14개, 각종 규칙과 상태 파일. 결과: 15K 토큰이 세션 시작과 동시에 소비되고, 실제 작업에 쓸 공간이 줄어들었다. 더 심각한 문제는 관련 없는 정보가 모델의 주의를 분산시킨다는 것이었다.

에이전트 24→15 축소, 스킬 14→9 축소를 거치면서 발견한 것:

> 컨텍스트는 “넣는 것”이 아니라 “빼는 것”으로 최적화된다.

Augment Code의 연구 — “smart retrieval into 200K context가 1M everything-dump보다 낫다” — 가 이 경험과 정확히 합치한다.

42K baseline에서 시작해 150K에 auto-compact하는 현재 정책은 이 교훈의 산물이다. Context as Currency — 컨텍스트의 모든 토큰에는 비용이 있고, 그 비용 이상의 가치를 제공하지 않는 토큰은 제거해야 한다.

이 발견의 이면에는 성장→과잉→축소→안정화의 4단계 패턴이 있다.

성장기: 에이전트 3→24개, 스킬 7→14개. 새 기능을 계속 추가한다. 시스템이 확장될수록 능력이 증가한다는 직관.

과잉기: 15K 토큰 자동 로딩, 8개 hook, 13개 skill, 35개 pipeline 규칙, 4-layer CE 아키텍처. 복잡성이 기하급수적으로 증가. 이 시점에서 Context Rot(Chroma 연구 개념)이 발생한다 — 컨텍스트에 낡거나 무관한 정보가 쌓이면 모델 성능이 오히려 떨어진다.

축소기: 에이전트 24→15(삭제 4 + 병합 10→5), 스킬 14→9, "경량 파이프라인" 분기 도입. Augment Code 연구와 실증적 합치.

안정기: 42K baseline → 150K auto-compact. 체인 결과 오프로딩(20-30K 절감). 시스템의 성숙은 기능을 추가하는 것이 아니라 불필요한 것을 제거하는 것으로 드러난다.

FINDING 05

AI와의 협업에는 프로토콜이 필요하다

처음에는 자유롭게 대화했다. 그러다 반복적으로 같은 실수가 발생했다. “Claude가 확인 없이 단정하는 패턴 반복: ‘품질 게이트 없다’(실제로 있었음), ‘section title 빈 칸’(heading 필드가 있었음), ‘Opus 4.6 미존재’(현재 모델임).” — 이건 실제 Failure 기록이다.

이 경험에서 나온 것이 프로토콜이다.

Phase v3: Diagnose → Architect → Build → Harden. 구현 전에 문제 정의를 강제하고, 구현 후에 검증을 강제한다.

PDR(Pipeline Decision Record): 모든 설계 결정에 문제/결정/이유/영향을 명시적으로 기록한다.

Foundation 3문서: 모든 프로젝트에 philosophy.md(왜), principles.md(뭘), workflow.md(어떻게)를 둔다.

> AI와의 협업 품질은 프롬프트의 단어가 아니라 판단의 전제를 구조화하는 프로토콜에 의해 결정된다.

FINDING 06

멀티 에이전트는 오케스트레이션 없이 카오스다

Claude Code + Codex + Gemini 3개 CLI를 병렬 운영하면서 배운 것이 있다. 각각은 유능하다. 그러나 조율 없이 동시에 돌리면 카오스가 된다.

실제 사고: “병렬 파일 쓰기 금지. 에이전트 4개가 같은 JSON을 동시 수정 시 충돌 발생 확인.” 그리고 “RemoteTrigger 82건 일괄 처리 실패 — 4건만 처리 후 세션 종료.”

이 경험에서 나온 인프라:

.ctx/ Cross-CLI 공유 메모리: CLI 간 작업 위임 시 컨텍스트 공유. rulesync: 규칙 SoT를 한 곳에서 관리하고 CLAUDE.md/GEMINI.md/AGENTS.md로 자동 배포. worktree 격리: 병렬 작업 시 Git worktree로 파일 시스템 격리.

> 멀티 에이전트 시스템의 핵심은 각 에이전트의 능력이 아니라 에이전트 간의 조율이다. 이건 인간 팀에서도 마찬가지지만, AI 에이전트에서는 더 극적이다 — 에이전트는 “상식적으로 알아서” 충돌을 피하지 않기 때문이다.

FINDING 07

빼기가 더하기보다 어렵고 중요하다

시스템이 커지면 복잡성이 기하급수적으로 증가한다. mcp-memory의 실제 Pattern 노드 중 하나: “에이전트 시스템의 성숙도: 더하기에서 빼기로 전환하며 효율성 극대화.”

구체적 이력: 에이전트 3개 → 24개 → 15개. 스킬 7개 → 14개 → 9개. Pipeline 규칙은 계속 증가 중이었으나 “경량 파이프라인” 분기를 만들어 소규모 작업은 규칙 면제.

과잉 설계의 위기가 있었다. 15K 토큰 자동 로딩, 8개 hook, 13개 skill, 35개 pipeline 규칙, 4-layer CE 아키텍처. 이걸 “Context Rot”(Chroma의 연구 개념)으로 진단했다 — 컨텍스트에 낡은 정보가 쌓이면 성능이 오히려 떨어진다.

> 시스템의 성숙은 기능을 추가하는 것이 아니라 불필요한 것을 제거하는 것으로 드러난다.

mcp-memory 도입 전후를 정성적으로 비교할 수 있다.

도입 전 (2026년 2월 이전): 매 세션이 백지 상태에서 시작. 이전 세션의 결정을 기억하려면 수동으로 마크다운 파일을 읽어야 했다. 같은 실수를 반복해도 발견할 메커니즘이 없었다. “어제 왜 그렇게 결정했지?”에 답하려면 채팅 로그를 뒤져야 했다.

도입 후: 세션 시작 시 L2+ 원칙이 자동 주입. 이전 Failure가 recall 최우선으로 떠오름. Decision 히스토리를 관계 그래프로 추적 가능. “이 결정은 어떤 원칙에서 나왔고, 그 원칙은 어떤 실패에서 배운 것인가?”를 graph traversal로 답할 수 있다.

정성적 차이: 세션 간 연속성이 생겼다. “매번 처음부터”가 “이어서”로 바뀌었다. 그리고 의외의 효과: 판단을 기록하는 행위 자체가 판단의 품질을 높인다 — “이걸 Decision으로 기록할 만큼 중요한가?”라는 질문이 경솔한 결정을 걸러낸다.

FINDING 08

자기 시스템을 측정하면 자기 사고가 보인다

auto-iterate 프로젝트(measure.py + policy engine)를 만들면서 드러난 것이 있다. L1-L5 레이어별 노드 분포를 측정하면 사고의 편향이 보인다.

L1(행동)에 편중 → 실행 편향. 생각하기 전에 행동부터 한다. L2(패턴)에 편중 → 분석 편향. 분석만 하고 결정을 미룬다. L3(원칙)이 빈약 → 매번 처음부터 판단한다. 축적이 안 된다.

현재 분포: L1이 가장 두껍고(Decision 987), L2가 그 다음(Insight 481 + Pattern 360), L3이 상대적으로 적다(Principle 304). 이건 "판단은 많이 하지만, 패턴 추출은 활발하고, 원칙 승격은 아직 부족하다"는 자기 진단이다.

> 보통 사람은 자기가 어떻게 생각하는지 모른다. 생각의 분포를 볼 수 없기 때문이다. 온톨로지의 타입 분포는 이 "보이지 않는 것"을 볼 수 있게 만든다.

FINDING 09

온톨로지는 거울이다

4,786개 노드의 분포가 보여주는 것은 시스템의 구조가 아니라 만든 사람의 사고 구조다.

Decision이 가장 많다 → 의사결정 중심으로 사고한다. Failure를 적극 기록한다 → 실패를 학습 자원으로 본다. Tool이 두 번째로 많다 → 실천적 지식을 중시한다. L4-L5가 비어 있다 → 가치/세계관 수준의 결정화가 덜 됐다. Narrative가 271개 → 경험을 이야기로 구조화하는 습관이 있다.

> 이건 설계 의도가 아니었다. 시스템을 쓰다 보니 자연스럽게 드러난 자화상이다. 온톨로지가 사용자를 비추는 거울이 되는 현상. 이 거울은 왜곡되지 않는다 — DB에는 수치만 있기 때문이다.

이것이 포트폴리오 관점에서도 강한 이유: 이 시스템은 “AI 메모리 도구”를 넘어 “인간이 자기 사고를 구조화하고 관찰할 수 있게 만드는 도구”다.

FINDING 10

만드는 행위 자체가 이해를 바꾼다

이 시스템을 만들기 전과 후에 “기억”, “판단”, “학습”에 대한 이해가 달라졌다.

들뢰즈를 읽었을 때, 현실화(actualisation)는 추상적 개념이었다. crystallized_into relation을 구현했을 때, 이것은 구체적 연산이 되었다 — Observation 노드에 Pattern 노드로의 엣지를 만들고, 원본의 confidence를 낮추는 코드.

BCM 가소성을 논문으로 읽었을 때, 이것은 시냅스의 수학적 모델이었다. confidence 필드와 활성화 횟수 기반 임계값으로 구현했을 때, 이것은 “얼마나 자주 이 지식을 다시 꺼내 쓰는가”에 대한 운영 정책이 되었다.

CLS(상보적 학습 시스템) 이론 — 해마의 빠른 학습과 신피질의 느린 통합 — 을 읽었을 때, 이것은 신경과학 모델이었다. Session injection(빠른 주입)과 ontology 승격(느린 통합)으로 구현했을 때, 이것은 “세션 안의 즉각적 맥락”과 “세션을 넘어서는 구조적 지식”을 분리하는 설계 원칙이 되었다.

> 구현이 이해를 바꾼다. 이론을 읽는 것과 이론을 코드로 옮기는 것은 다른 종류의 이해를 낳는다. 읽기는 개념을 갖게 하지만, 만들기는 개념의 한계와 가능성을 동시에 드러낸다.

한계도 있다. 첫째, 대조군이 엄밀하지 않다. “도입 전후 비교”는 정성적이지 정량적이 아니다. 같은 기간에 사용자의 숙련도도 올라갔기 때문에, 개선이 시스템 때문인지 숙련 때문인지 분리하기 어렵다. 둘째, 이 시스템은 N=1이다. 한 사람의 경험에서 일반적 결론을 도출하는 것의 한계를 인정해야 한다. 셋째, “온톨로지는 거울”이라는 발견은 동시에 위험이다 — 거울이 편향을 반영하면, 시스템이 편향을 강화할 수 있다.

COLOPHON

Beta — 무엇이 드러났는가
mcp-memory System Analysis, Part II of III

Paul Park × Claude
2026-03-28

Typeset in Inter, Noto Sans KR, and Noto Serif KR.
Body 10.5pt / 16pt leading on 145mm measure.
Page format 247 × 340mm.

El Croquis Design System — Digital PDF v2.0